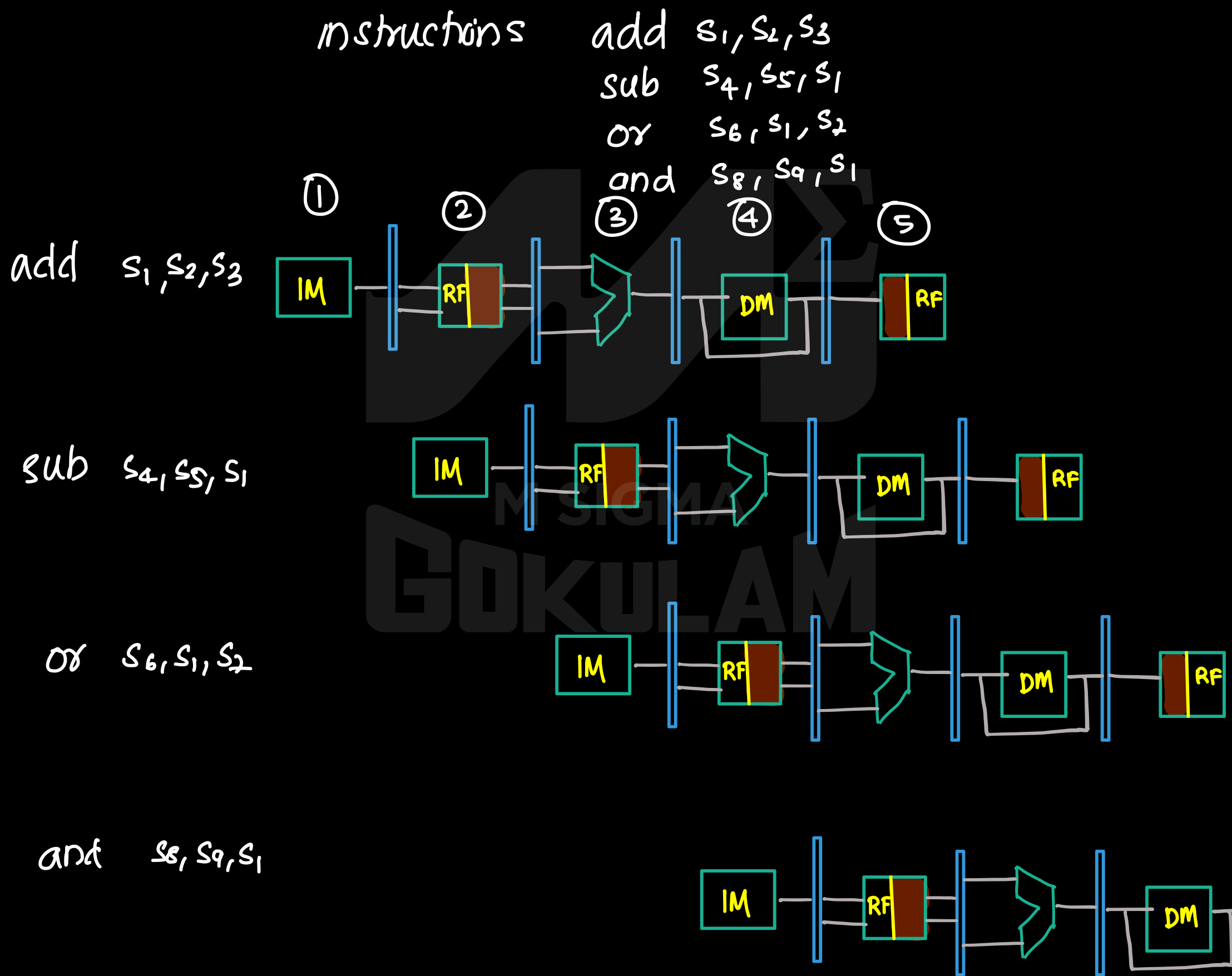


Pipeline hazards. (Note)

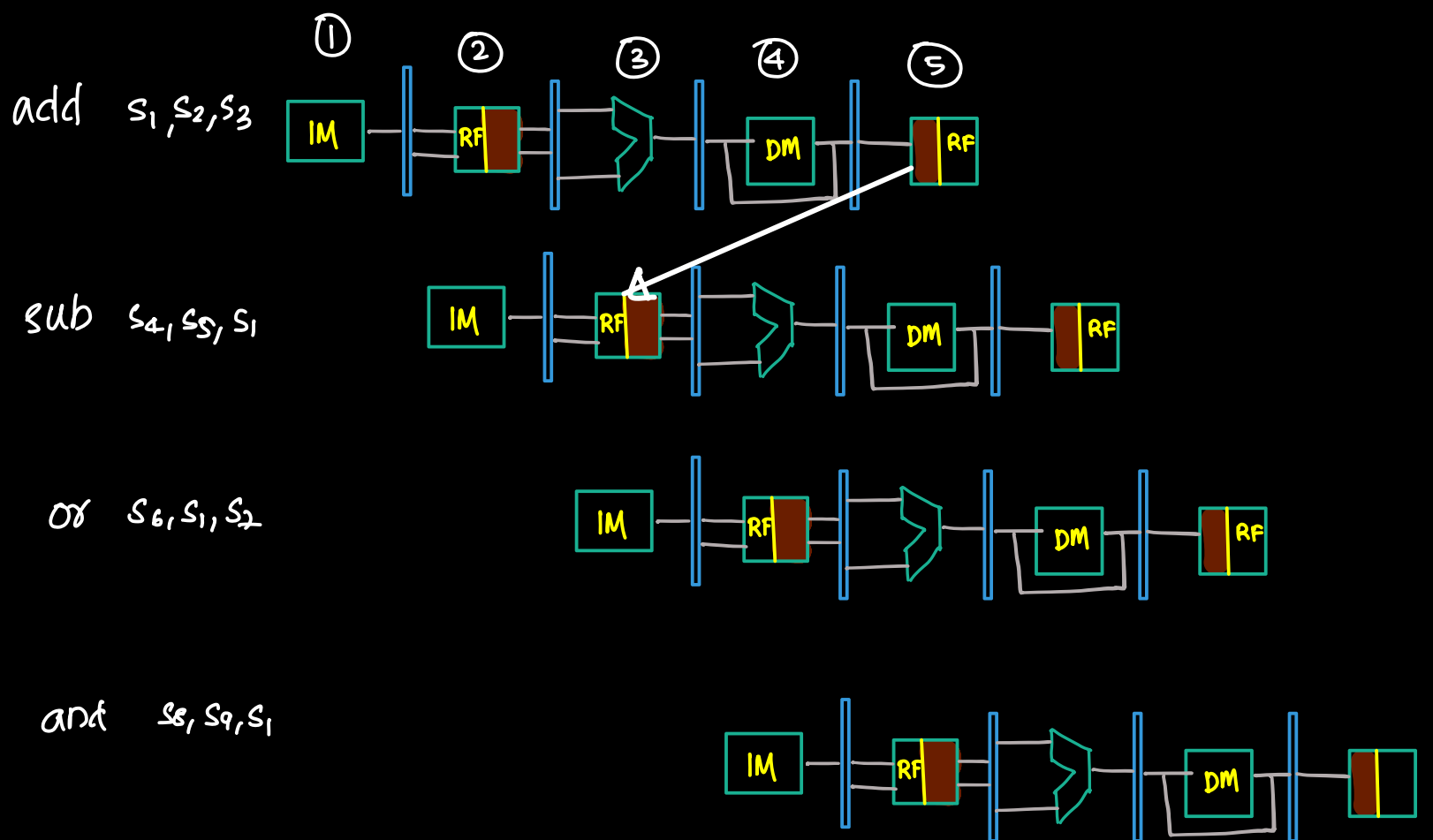
— In a pipelined processor, instruction execution is divided into different stages. and multiple instructions are executed simultaneously by overlapping different stages. This introduces a problem called pipeline hazards.

→ Consider a 4 instructions executed simultaneously using 5-stage pipelining.



① Add instruction store the sum in cycle 5., i.e. result of 1st instruction available at end of 5th clock cycle. But 2nd instruction (sub) reads s_1 in cycle 3. Register s_1 contains the result of add instruction only at the end of 5th clock cycle. So and instruction (sub) reads wrong.

value of s_i . this create a problem. this is called hazards. (N)



similarly 3rd instruction (or) reads the content of s_1 in cycle 4. But correct value is not available. this will cause hazard.

Here one instruction that depends on the result of another instruction that has not yet completed.

pipeline hazards

When one instruction is dependent on the result of another that has not yet completed, a hazard occurs.

Different types hazards

- ① Data hazard
- ② Control hazard.

① Data hazard

②

Data hazard occurs in a pipelined processor when an instruction depends on the result of a previous instruction that has not yet completed execution. To ensure correct program execution, data hazard must be detected and resolved using different methods.

Methods to solve data hazards

- ① solving data hazards with forwarding
- ② solving data hazards with stalls

① solving data hazards with forwarding

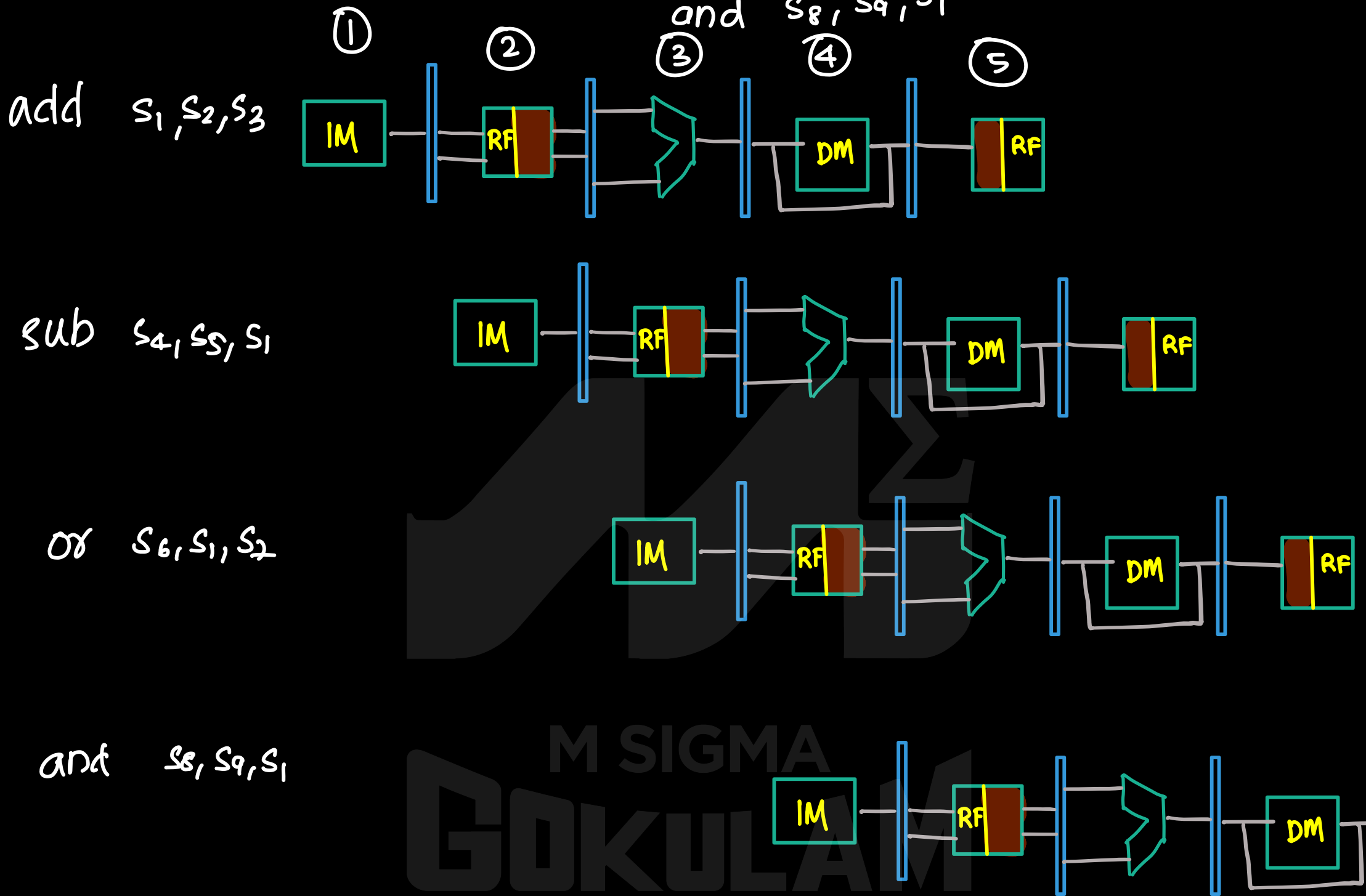
— Forwarding is a technique in which result of an instruction is sent directly from a later pipeline stage to an earlier stage without waiting for it to be written back to the register.

— Forwarding a result from memory or write back stage to a dependent instruction in the execute stage.

M SIGMA
GOKULAM

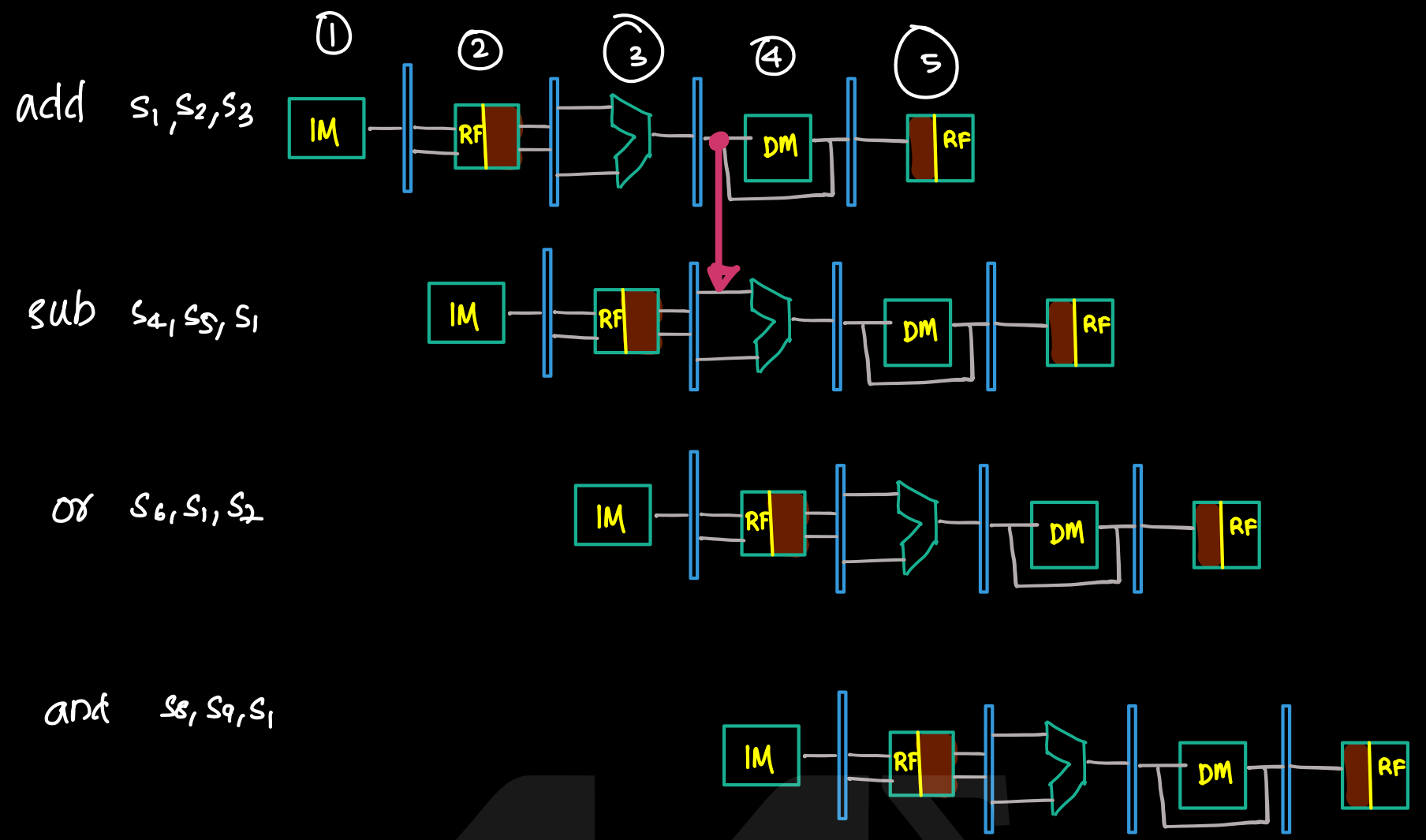
→ Consider a 4 instructions executed simultaneously using 5-stage pipelining. (N)

Instructions add s_1, s_2, s_3
 sub s_4, s_5, s_1
 or s_6, s_1, s_2
 and s_8, s_9, s_1



① Add instruction store the sum in cycle 5, i.e. result of 1st instruction available at end of 5th clock cycle. But 2nd instruction (sub) reads s_1 in cycle 3. Register s_1 contains the result of add instruction only at the end of 5th clock cycle. So 2nd instruction (sub) reads wrong value of s_1 . This creates a problem. This can be solved by forwarding.

→ In cycle 4, result of 1st instruction forwarded from memory stage to execution state of sub instruction.



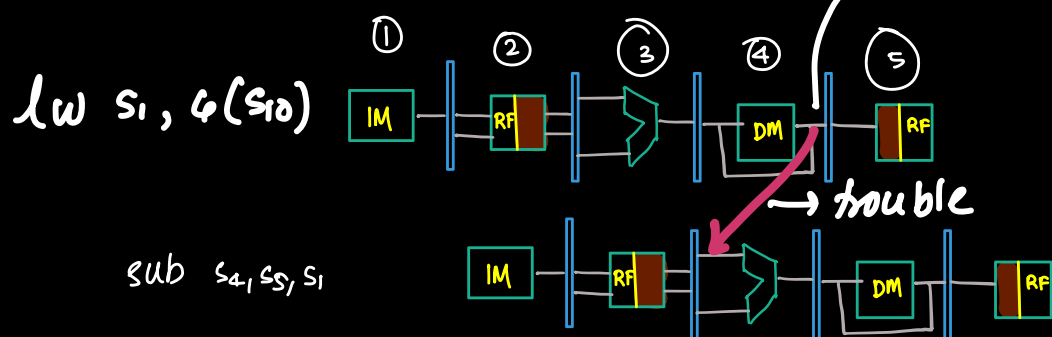
② solving data hazards with stalls

(N)

consider a load instruction

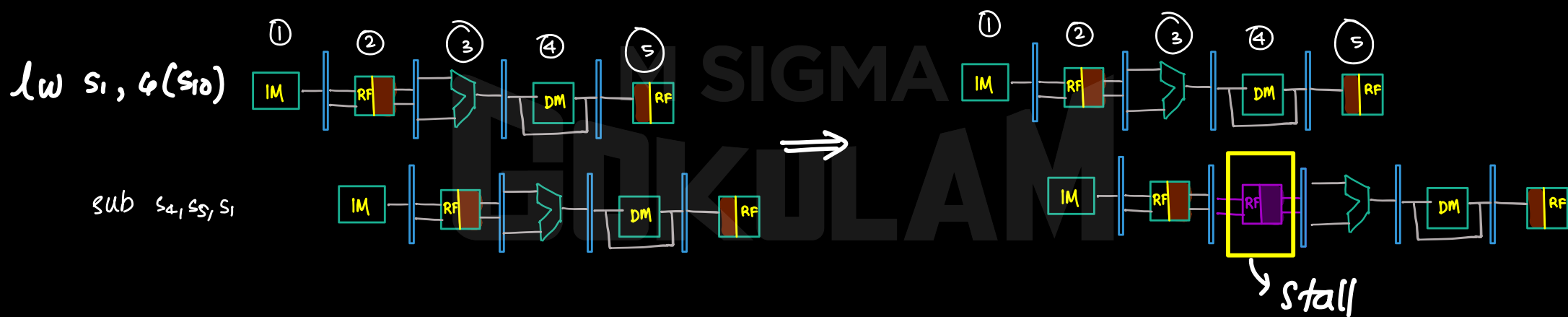
eg: $lw\ s_1, 4(s_{10})$

- load the content of memory location to the register s_1 .
- this instruction finish reading data at the end of memory stage.



Content of memory location will be available at the end of memory stage. But sub instruction need the content of s_1 in the beginning of 4th clock cycle. so data cannot be forward to execute state of sub instruction.

This problem can be solved by holding up operation until the data is available.



→ Now Result of 'lw' instruction available at the end of memory stage (4th cycle). and sub instruction needs result of 'lw' in 5th clock cycle.

Control Hazards.

(2)

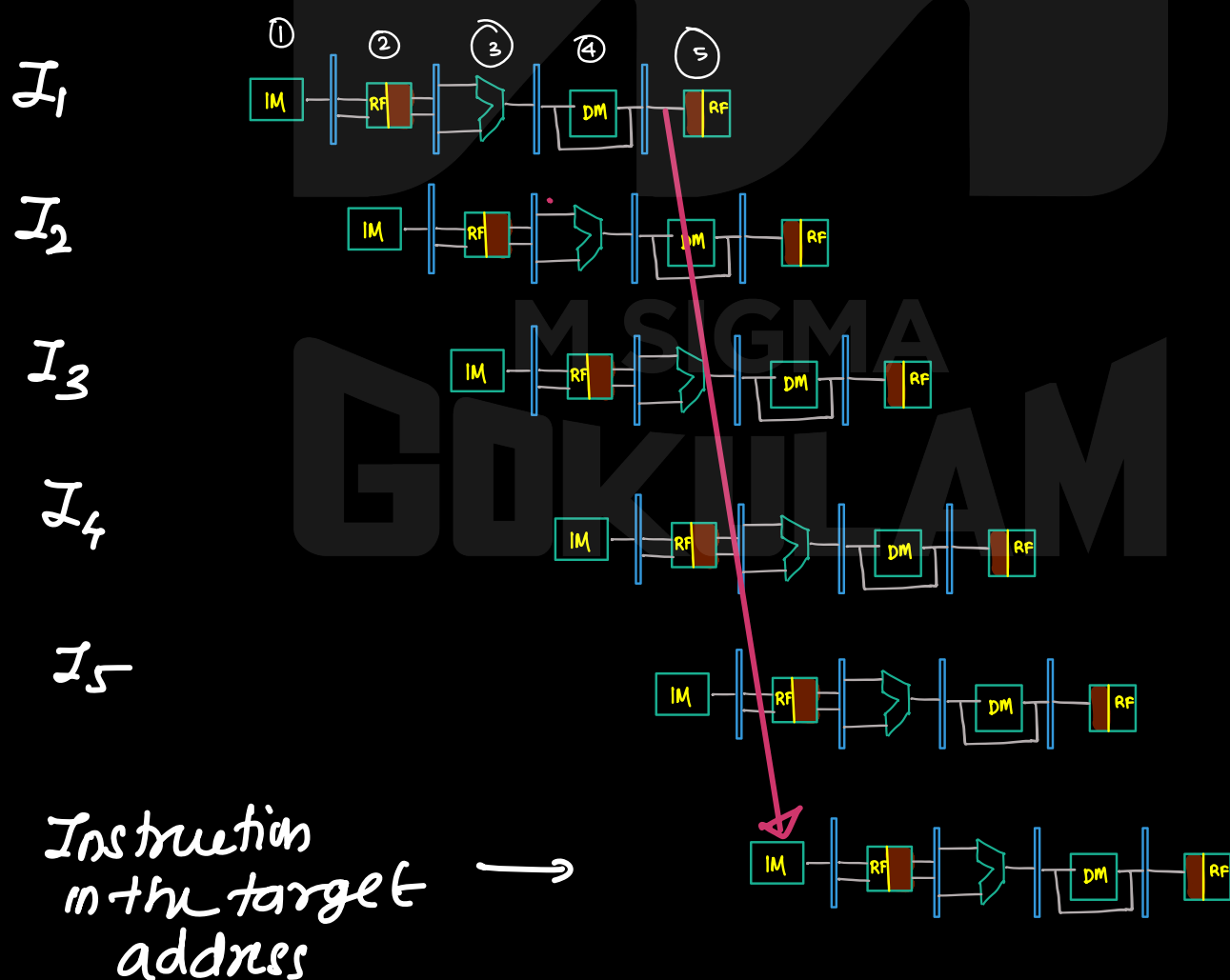
- In pipelined processor, control hazards occurs due to the branch instructions. Because processor does not know what instruction to fetch next, branch target address is not calculated at time of next instruction is fetched

→ Consider a 5 instructions I_1, I_2, I_3, I_4, I_5 , in which I_1 is the branch instruction.

Eg: $I_1 \rightarrow \text{beq } s_1, s_2, L_1$
branch if content of s_1 & s_2 are equal
 $L_1 \rightarrow$ target address.

- $I_1 \rightarrow$ Branch instruction
 $I_2, I_3, I_4, I_5 \rightarrow$ sequential instruction.

Pipe line execution



- Assume that branch target address is available at the beginning of the 5th clock cycle

— clock wise progress

(N)

Cycle-1 : I_1 is fetched

Cycle-2 : I_1 decoded, I_2 Fetched

Cycle-3 : I_1 executed, I_2 Decoded, I_3 Fetched

Cycle-4 : I_1 Memory stage, I_2 executed, I_3 Decoded

Cycle-5 : I_1 write back stage (Branch target address)

By the beginning of cycle 5, the processor has already fetched I_2, I_3, I_4, I_5 (4 instructions)

→ When branch is taken, then processor execute the instruction in the target address and flushed the already fetched wrong path instruction (I_2, I_3, I_4, I_5).

→ So mispenalty = 4 cycle (Because 4 instruction must be flushed.)

How to reduce this mispenalty.

— To reduce the number of flushed instructions, we try to predict branch target earlier

— If the branch target available in cycle 3 (execution stage of I_1). then fewer instructions fetched wrongly. Hence flush count reduces.

